

Memoria en Agentes LLM

Módulo 10 · Sistemas Agénticos y Multi-Agente

Cuatro tipos de memoria, compresion del historial, mem0, Zep y evaluacion de la calidad de la memoria.

1. Introducción · 2. Qué es · 3. Cómo funciona · 4. Ejemplos reales
5. Errores comunes · 6. Aplicación práctica · 7. Relación con módulos · 8. Resumen ejecutivo

Guía de estudio detallada · +2.000 palabras

Memoria en Agentes LLM

El problema fundamental de los LLMs es la apatridia: cada llamada al API es independiente y el modelo no recuerda nada de las interacciones anteriores. Para un chatbot conversacional, esto se resuelve enviando el historial de la conversación en cada llamada. Para un agente de larga duración que trabaja durante horas o días, esto es insuficiente: la ventana de contexto se llena, las conversaciones de semanas atrás se pierden, y el agente no puede aprender de sus errores pasados.

La memoria en agentes LLM es el conjunto de mecanismos que permiten al agente mantener coherencia a lo largo del tiempo, más allá de los límites de la ventana de contexto. En 2025, la investigación en memoria de agentes ha crecido exponencialmente: más de 1.200 papers publicados en 2024 sobre el tema, con frameworks como A-Mem (2025), mem0, y Zep emergiendo como soluciones de producción. La memoria es ahora reconocida como el "cornerstone" arquitectónico de los sistemas agénticos efectivos.

Los cuatro tipos de memoria en agentes

Memoria de trabajo (Working memory)

La memoria de trabajo es la ventana de contexto actual del LLM: todo lo que el agente "ve" en el momento presente. Incluye el objetivo, el historial de la sesión actual, los resultados de las herramientas ejecutadas, y el contexto RAG recuperado. Es la más rápida de acceder (ya está en el contexto) pero la más limitada en capacidad (típicamente 16K-200K tokens). La gestión de la memoria de trabajo (qué incluir, qué truncar, cómo comprimir) es el desafío operativo más inmediato.

Memoria episódica (Episodic memory)

La memoria episódica almacena el historial de interacciones y eventos pasados: qué hizo el agente en sesiones anteriores, qué funcionó y qué no, qué conversaciones tuvo con el usuario. Es análoga a la memoria autobiográfica humana. En sistemas de agentes, se implementa almacenando resúmenes de sesiones pasadas en una base de datos con búsqueda semántica. Cuando el agente necesita contexto de interacciones pasadas, recupera los episodios más relevantes y los incluye en la ventana de contexto.

Memoria semántica (Semantic memory)

La memoria semántica es el conocimiento general sobre el dominio: hechos, conceptos, relaciones, y procedimientos que el agente ha aprendido a lo largo del tiempo. En agentes LLM, la memoria semántica puede ser: paramétrica (conocimiento incorporado en los pesos del modelo mediante fine-tuning) o no paramétrica (conocimiento almacenado externamente y recuperado con RAG). La memoria semántica no paramétrica es más fácil de actualizar y auditar.

Memoria procedimental (Procedural memory)

La memoria procedimental almacena cómo hacer cosas: procedimientos, estrategias, y heurísticas aprendidas de la experiencia. En agentes LLM, puede implementarse como: prompts de sistema que capturan las mejores prácticas aprendidas (el agente actualiza su propio system prompt con lecciones aprendidas), ejemplos few-shot de alta calidad derivados de éxitos pasados (el agente recuerda qué tipo de respuesta funcionó bien en situaciones similares), o herramientas especializadas que implementan procedimientos que el agente ha aprendido a usar efectivamente.

SECCIÓN 3 · CÓMO FUNCIONA

Estrategias de compresión y gestión de la memoria

Compresión del historial de conversación

Para agentes de larga duración, el historial de conversación puede saturar la ventana de contexto. Las estrategias de compresión son: resumen periódico (cada N turnos, comprimir el historial en un resumen conciso), ventana deslizante (mantener solo los últimos K turnos), y extracción de hechos clave (extraer solo la información factualmente relevante del historial y descartar el contexto conversacional). La estrategia óptima depende del tipo de tarea: para tareas de análisis, los hechos clave son más importantes que los detalles conversacionales; para tareas de asistencia personal, el tono y las preferencias del usuario son tan importantes como los hechos.

Memoria jerárquica: múltiples niveles de abstracción

La memoria jerárquica organiza la información en múltiples niveles de granularidad: memoria detallada (los steps exactos y resultados de las últimas K iteraciones), memoria de resumen (resumen de los objetivos y resultados de cada sesión completada), y memoria de alto nivel (patrones y preferencias aprendidas del usuario a lo largo de múltiples sesiones). El agente recupera del nivel apropiado según la granularidad necesaria para la tarea actual.

Conflict resolution: qué hacer cuando las memorias son contradictorias

Las memorias en sistemas agénticos pueden ser contradictorias: el agente recordó que el usuario prefería informes en formato PDF, pero en la última sesión el usuario pidió formato Word. El sistema de memoria debe resolver estos conflictos de forma inteligente: priorizar las memorias más recientes para preferencias que pueden cambiar, mantener ambas memorias para hechos que pueden coexistir, y alertar al agente cuando detecta una contradicción para que pueda clarificar.

SECCIÓN 4 · EJEMPLOS REALES

- mem0 en asistente de ventas B2B: el agente almacena en mem0 la información descubierta sobre cada cliente (presupuesto, timeline, decisores, objeciones habituales, comunicaciones anteriores). En cada nueva interacción, recupera el contexto relevante del cliente. El vendedor no tiene que repetir el contexto cada vez; el agente ya sabe el historial de la oportunidad. Resultado: conversaciones más eficientes y mejor tasa de conversión.
- Memoria procedimental en agente de coding: el agente almacena los patrones de código que ha producido y que han sido aceptados vs rechazados por el equipo en revisiones de

código. Con el tiempo, la memoria procedimental del agente se alinea con el estilo de código del equipo sin necesidad de fine-tuning adicional.

- Zep para memoria de soporte al cliente: Zep almacena el historial completo de interacciones de cada cliente (tickets anteriores, problemas conocidos, soluciones que funcionaron, preferencias de comunicación). El agente de soporte recupera el historial relevante antes de cada interacción, produciendo respuestas más personalizadas y evitando soluciones ya intentadas.
- Generative agents (Park et al., 2023): uno de los estudios más influyentes en memoria de agentes. Simulaciones de agentes sociales con memoria episódica que se comprime en reflexiones de alto nivel. Los agentes exhiben comportamientos emergentes coherentes (planificación de actividades diarias, relaciones sociales) gracias a su arquitectura de memoria jerárquica.

SECCIÓN 5 · ERRORES Y MALENTENDIDOS COMUNES

Error 1: Almacenar todo el historial sin filtrado. No toda la información de una sesión merece ser almacenada en memoria a largo plazo. Almacenar demasiado produce ruido que degrada la calidad de la recuperación futura. Implementa criterios de selección: almacena solo la información con alta importancia o alta probabilidad de ser relevante en el futuro.

Error 2: Recuperar siempre las mismas K memorias más recientes sin considerar la relevancia. La memoria más reciente no es siempre la más relevante para la tarea actual. Usa búsqueda semántica para recuperar las memorias más relevantes por contenido, no solo por recencia.

Error 3: No gestionar la privacidad y la seguridad de la memoria de usuario. Las memorias del agente pueden contener información sensible del usuario. Implementa controls de acceso (el agente de un usuario no puede acceder a las memorias de otro), cifrado en reposo, y política de retención y eliminación de memorias.

SECCIÓN 6 · APLICACIÓN PRÁCTICA

Para implementar memoria persistente con mem0: pip install mem0ai. Inicializa el cliente con el modelo de embedding y el backend de almacenamiento. Para añadir una memoria: client.add(messages=[{"role": "user", "content": "Prefiero informes en PDF"}], user_id="user_123"). Para recuperar memorias relevantes: memories = client.search(query="formato de informe preferido", user_id="user_123"). Para el agente, incluye las memorias relevantes en el system prompt al inicio de cada sesión.

Para la compresión del historial en agentes de larga duración: implementa un timer de compresión que se activa cada N iteraciones. El compressor es un prompt al LLM: "Dado el siguiente historial de conversación [historial], genera un resumen conciso (máximo 500 tokens) que preserve: (1) los hechos clave descubiertos, (2) las decisiones tomadas y su justificación, (3) los pending items que aún no se han completado, (4) las preferencias del usuario expresadas." Reemplaza el historial completo con el resumen en la ventana de contexto.

SECCIÓN 7 · RELACIÓN CON OTROS MÓDULOS

- M7 (RAG): la memoria semántica no paramétrica se implementa con los mismos principios que el RAG.
- M10-T2 (Arquitectura de agentes): la memoria persistente es un componente arquitectónico de agentes avanzados.
- M4-T6 (Fine-tuning): la memoria paramétrica (conocimiento en los pesos del modelo) se actualiza con fine-tuning.

SECCIÓN 8 · RESUMEN EJECUTIVO

Los cuatro tipos de memoria en agentes son: de trabajo (ventana de contexto actual), episódica (historial de sesiones pasadas), semántica (conocimiento del dominio, implementada con RAG), y procedimental (cómo hacer cosas, patterns aprendidos). La gestión de la memoria de trabajo requiere compresión periódica del historial (resumen cada N iteraciones) y ventana deslizante. Las herramientas de referencia: mem0 (memoria a largo plazo con búsqueda semántica), Zep (memoria de usuario para sistemas conversacionales). La memoria jerárquica (detallada → resumen → alto nivel) es el patrón más efectivo para agentes de larga duración. Privacy y control de acceso son imprescindibles para memorias de usuario.